PRD

Arch ive Dashboard

레이어 아키텍처 위반 추적 & 개선 현황 서비스

버전	v1.0
작성일	2026년
작성자	LLM Platform Team

1. 개요 (Overview)

1.1 배경 및 문제 정의

소프트웨어 프로젝트가 성장할수록 레이어 아키텍처 규칙(presentation → domains → infrastructure → data_access)을 지키는 것이 어려워진다. 개발자들이 빠른 기능 구현을 위해 "일단" 금지된 방향으로 import를 추가하고, 이것이 누적되면 구조적 부채(structural debt)로 이어진다.

> 문제 현재 팀에는 코드 병합(PR) 시점에 아키텍처 위반을 정량적으로 추적하고, 개선 기여를 가시화하는 도구가 없다.

1.2 제품 비전

"PR 단위로 아키텍처 위반을 추적하고, 레이어별 위반 비중과 기여자별 개선 현황을 실시간 대시보드로 제공하여 팀이 구조적 부채를 능동적으로 관리할 수 있게 한다."

1.3 목표

- CI 파이프라인(GitHub Actions)에서 자동으로 레이어 위반을 감지하고 기록
- 레이어 경계별 위반 비중을 정량적으로 파악 (어느 경계가 가장 많이 뚫리나)
- 기여자별 개선 현황을 누적 delta 기준으로 랭킹화
- 팀 전체 violation 추이를 시계열로 시각화
- 내부망 전용 · 별도 인증 없이 즉시 운영 가능

1.4 대상 사용자

역할	사용 목적	주요 관심 지표
개발자 (PL·팀원)	내 PR이 아키텍처를 얼마나 개선했는지 확인	PR delta, 개인 랭킹
팀 리더 / PL	전체 아키텍처 부채 추이 모니터링	추이 차트, 레이어별 비중
코드리뷰어	PR 리뷰 시 위반 현황 참조	체크 이력, 위반 상세

2. 기능 요구사항 (Functional Requirements)

2.1 CI 연동 — 위반 감지 및 수집

FR-01: import-linter 실행

- GitHub Actions PR 트리거 시 자동 실행
- 분석 대상: 프로젝트 루트의 모든 Python 모듈
- .importlinter 설정 파일 기준으로 레이어 계약 검증

FR-02: 결과 파싱 및 전송

- import-linter JSON/텍스트 출력 파싱
- 위반 항목별 from_module, to_module, from_layer, to_layer 추출
- PR 메타데이터(번호, 작성자, 브랜치, commit SHA)와 함께 API POST
- 전송 실패 시 exit code 1 반환 (CI 실패 처리 가능)

FR-03: delta 자동 산출

- 직전 동일 repo 체크 결과를 baseline으로 delta = 현재 - 이전 계산
- delta < 0 : 개선 / delta > 0 : 악화 / delta = 0 : 유지

2.2 API 서버

FR-04: 체크 결과 등록 (POST /api/checks/)

- 요청 페이로드: repo, author, branch, commit_sha, pr_number, violation_count, violations[]
- violations 배열: { from_module, to_module, from_layer, to_layer }
- 저장 시 delta 자동 계산 및 저장

FR-05: 레이어 경계별 위반 비중 (GET /api/stats/layer-violations)

- 집계: (from_layer, to_layer) 쌍별 위반 건수
- 비중 = 해당 쌍 건수 / 전체 위반 건수 × 100 (%)
- 내림차순 정렬

FR-06: 기여자별 개선 랭킹 (GET /api/stats/authors)

- 집계: author별 delta 합산, PR 수, 최고 단일 개선치
- total_improved 오름차순 정렬 (음수일수록 개선 기여 큼)

FR-07: 위반 추이 (GET /api/stats/trend)

- 최근 N개 체크의 (checked_at, violation_count, author, delta) 시계열 반환

FR-08: 전체 요약 (GET /api/stats/summary)

- total_checks, peak_violations, best_violations, latest_violations, contributor_count

2.3 대시보드 UI

FR-09: 요약 카드 **(4개)**

- 현재 위반 수 (최신 체크 기준)
- 역대 최저 위반 수
- 총 체크 횟수 / 기여자 수
- 최고 기여자 이름 + 개선 건수

FR-10: 위반 수 추이 차트

- X축: 체크 시각, Y축: violation_count
- 개선된 점 = 초록, 악화된 점 = 빨강
- 툴팁: author, delta 표시

FR-11: 레이어 경계별 위반 비중 테이블

- FROM 레이어 → TO 레이어, 건수, 비중(%) + 비율 바
- 레이어별 색상 구분 배지

FR-12: 기여자 개선 랭킹

- 랭킹, 아바타, 이름, PR 수, 최고 단일 개선, 누적 delta
- 1위 별(★) 강조

FR-13: 최근 체크 이력 테이블

- PR번호, 작성자, 위반 수, delta 배지(▼ 개선 / ▲ 악화), commit SHA, 시각

FR-14: repo 필터

- 상단 입력창에 owner/repo 입력 시 해당 repo 데이터만 조회
- 빈 값 = 전체 repo 합산
- 30초 자동 갱신

3. 비기능 요구사항 (Non-Functional Requirements)

분류	항목	요구 수준
성능	API 응답 시간	p95 < 300ms (SQLite 기준, 데이터 10만 건 이하)
가용성	서버 업타임	업무 시간 내 99% 이상 (내부망 단일 서버)
보안	인증	내부망 전용, 별도 인증 없음. 외부 노출 금지
확장성	DB 교체	SQLite → PostgreSQL 전환 시 database.py만 수정
운영	로그	Uvicorn 표준 로그, 에러 시 stderr 출력
호환성	Python	3.11 이상
CI	Actions 러너	self-hosted 내부망 러너 기준

4. 시스템 아키텍처

4.1 전체 흐름

플로우 PR 생성 → GitHub Actions 트리거 → import-linter 실행 → report_arch.py 파싱·전송 → FastAPI 서버 수신·저장 → 대시보드 조회

4.2 컴포넌트 구성

컴포넌트	기술	역할
CI Script	Python 3.11	import-linter 실행·파싱·API 전송
API Server	FastAPI + Uvicorn	결과 수신, 통계 집계, HTML 서빙
Database	SQLite (WAL 모드)	체크 이력 + 위반 상세 저장
Dashboard	단일 HTML + Chart.js	실시간 대시보드 UI
CI/CD	GitHub Actions	PR 트리거 자동 실행

4.3 디렉터리 구조

```
arch-dashboard/
├── app/
│   ├── main.py              # FastAPI 진입점 + lifespan
│   ├── database.py          # SQLite 초기화, 커넥션
│   ├── schemas.py           # Pydantic 입출력 모델
│   ├── routers/
│   │   ├── checks.py        # POST·GET /api/checks
│   │   ├── stats.py         # GET /api/stats/*
│   │   └── dashboard.py     # GET / (HTML)
│   └── static/index.html    # 대시보드 SPA
├── scripts/
│   ├── report_arch.py       # CI 전송 스크립트
│   └── seed_demo.py         # 데모 데이터 생성
├── .github/workflows/
│   └── arch-check.yml
├── .importlinter            # 레이어 계약 정의
└── requirements.txt
```

5. 데이터 모델

5.1 arch_checks 테이블

컬럼	타입	설명
id	INTEGER PK	자동 증가
repo	TEXT	owner/repo 형식
pr_number	INTEGER?	PR 번호 (push 이벤트 시 NULL)
branch	TEXT	브랜치명
commit_sha	TEXT	커밋 해시
author	TEXT	GitHub 사용자명
checked_at	TEXT	ISO 8601 datetime
violation_count	INTEGER	이번 체크 위반 수
prev_violation_count	INTEGER?	직전 체크 위반 수 (baseline)
delta	INTEGER?	violation_count - prev (음수=개선)
raw_result	TEXT?	import-linter 원본 JSON

5.2 layer_violations 테이블

컬럼	타입	설명
id	INTEGER PK	자동 증가
check_id	INTEGER FK	arch_checks.id 참조
from_layer	TEXT	위반 출발 레이어명
to_layer	TEXT	위반 도착 레이어명
from_module	TEXT	위반 출발 모듈 전체 경로
to_module	TEXT	위반 도착 모듈 전체 경로

6. API 명세

Method	Endpoint	설명	주요 파라미터
POST	/api/checks/	체크 결과 등록	Body: ArchCheckRequest
GET	/api/checks/	체크 이력 목록	?repo=&limit=
GET	/api/checks/{id}	체크 상세 + 위반 목록	path: id
GET	/api/stats/layer-violations	레이어별 위반 비중	?repo=
GET	/api/stats/authors	기여자 개선 랭킹	?repo=
GET	/api/stats/trend	위반 수 시계열	?repo=&limit=
GET	/api/stats/summary	전체 요약 수치	?repo=
GET	/	대시보드 HTML	—

7. 구현 로드맵

Phase 1 — MVP (완료)

완료 범위 FastAPI 서버, SQLite DB, CI 스크립트, 대시보드 HTML, GitHub Actions workflow

- POST /api/checks/ — 결과 수신·저장·delta 산출
- GET /api/stats/* — 4종 통계 API
- 대시보드 — 요약 카드, 추이 차트, 레이어 비중, 기여자 랭킹, 체크 이력
- GitHub Actions workflow + report_arch.py CI 스크립트

Phase 2 — 운영 안정화

목표 실제 팀 사용 환경에 배포하고 피드백 반영

- Docker Compose 패키징 (서버 + DB 볼륨)
- PostgreSQL 마이그레이션 옵션 제공
- Slack/Mattermost 위반 증가 알림 웹훅
- PR 코멘트 자동 등록 (GitHub API 연동)

Phase 3 — 고도화

목표 멀티 레포 지원 및 분석 고도화

- 레포별·브랜치별 베이스라인 분리 관리
- 위반 모듈 상세 드릴다운 뷰
- 주간/월간 리포트 자동 생성 (PDF 이메일)
- 프론트엔드(JS/TS) dependency-cruiser 연동
- 팀 목표 violation 수 설정 및 달성률 트래킹

8. 설치 및 운영 가이드

8.1 서버 실행

```
# 의존성 설치
pip install -r requirements.txt

# 서버 실행 (포트 8080)
python -m app.main

# 대시보드 접속
open http://localhost:8080
```

8.2 GitHub Actions 설정

1. Repository Settings → Variables → New variable
2. Name: ARCH_DASHBOARD_URL, Value: http://내부서버IP:8080
3. .github/workflows/arch-check.yml 을 프로젝트에 추가
4. .importlinter 를 프로젝트 루트에 추가 (레이어 계약 정의)
5. PR 생성 시 자동 실행 확인

8.3 .importlinter 설정 예시

```
[importlinter]
root_package = app

[importlinter:contract:layered-architecture]
name = Layered Architecture
type = layers
layers =
    presentation
    domains
    infrastructure
    data_access
containers =
    app
```

9. 미결 사항 (Open Questions)

#	질문	현재 가정	결정 필요 시점
1	CI 위반 시 PR merge 차단 여부	continue-on-error: true (경고만)	Phase 2 시작 전
2	delta 베이스라인 기준 (main? 직전 PR?)	직전 동일 repo 체크	팀 합의 필요
3	멀티 레포 지원 우선순위	단일 repo MVP 우선	Phase 2
4	DB 용량 한계 대응 (오래된 이력 삭제)	미정 (SQLite 제한 없음)	Phase 2
5	Slack 알림 조건 (항상? delta>N?)	미정	Phase 2

— 문서 끝 —